

HelpDesk API - Diseño y Análisis de Requerimientos

Sistema de Gestión de Tickets y Mesa de Ayuda — Documento de Diseño y Análisis de Requerimientos

Agosto 2026

- [1. Resumen del Proyecto](#)
 - [1.1 Stack Tecnológico](#)
 - [1.2 Problema que Resuelve](#)
- [2. Análisis de Requerimientos](#)
 - [2.1 Actores del Sistema](#)
 - [2.2 Requerimientos Funcionales](#)
 - [2.3 Reglas de Negocio](#)
 - [2.4 Requerimientos No Funcionales](#)
- [3. Roles y Permisos](#)
- [4. Flujo de Estados del Ticket](#)
- [5. Prioridades](#)
- [6. Categorías](#)
- [7. Departamentos](#)
- [8. Modelo de Base de Datos](#)
 - [8.1 Tablas del Sistema](#)
 - [8.2 users](#)
 - [8.3 roles](#)
 - [8.4 user_roles](#)
 - [8.5 departments](#)
 - [8.6 categories](#)
 - [8.7 tickets \(tabla principal\)](#)
 - [8.8 ticket_comments](#)
 - [8.9 ticket_attachments](#)
 - [8.10 ticket_assignments](#)
 - [8.11 ticket_history](#)
 - [8.12 sla_policies](#)
 - [8.13 notifications](#)
 - [8.14 audit_logs](#)
 - [8.15 Relaciones del Modelo](#)
- [9. API REST](#)
 - [9.1 Autenticación](#)
 - [9.2 Usuarios](#)
 - [9.3 Tickets](#)
 - [9.4 Comentarios](#)
 - [9.5 Historial](#)
 - [9.6 Asignación](#)
 - [9.7 Filtros](#)
 - [9.8 Búsqueda](#)
 - [9.9 Dashboard](#)
- [10. DTOs](#)
- [11. Arquitectura](#)
 - [11.1 Paquete base](#)
 - [11.2 Estructura de un módulo \(ejemplo: department\)](#)
 - [11.3 Módulos del sistema](#)
 - [11.4 Convención de nombres por capa](#)
- [12. Seguridad](#)
- [13. Manejo de Errores](#)
- [14. Validaciones](#)
- [15. Transacciones](#)
- [16. Paginación](#)
- [17. Pruebas \(Tests\)](#)
 - [17.1 Pruebas Unitarias](#)
 - [17.2 Pruebas de Integración](#)
- [18. Docker](#)
- [19. Flyway](#)
- [20. Swagger](#)
- [21. Actuator](#)
- [22. Logging](#)
- [23. Roadmap de Evolución](#)
 - [23.1 Segunda Etapa: Redis](#)

1. Resumen del Proyecto

HelpDesk API es un sistema de gestión de tickets y mesa de ayuda pensado para empresas. El objetivo es construirlo como un **proyecto real**, no como un simple CRUD: incluye usuarios, roles, agentes, tickets, prioridades, categorías, comentarios, asignaciones, SLA, historial y auditoría.

El enfoque recomendado es empezar como **monolito modular** con Java 21 + Spring Boot 3, con una arquitectura que permita evolucionar más adelante hacia microservicios.

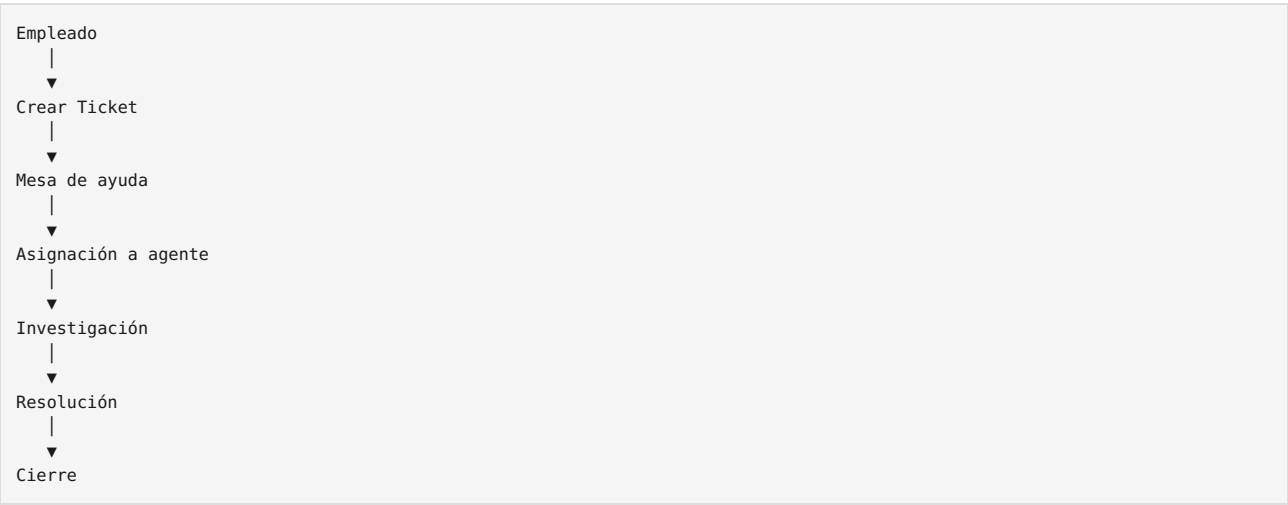
1.1 Stack Tecnológico

Categoría	Tecnologías
Lenguaje / Framework	Java 21, Spring Boot 3
Web / Persistencia	Spring Web, Spring Data JPA
Seguridad	Spring Security, JWT
Base de datos	PostgreSQL, Flyway
Validación / Mapeo	Bean Validation, MapStruct, Lombok
Documentación	OpenAPI / Swagger
Pruebas	JUnit 5, Mockito, Testcontainers
Infraestructura	Docker, Docker Compose

Opcionales para una segunda etapa: Redis, Kafka, WebSocket, Prometheus, Grafana.

1.2 Problema que Resuelve

Una empresa con 500 empleados recibe hoy sus incidencias por correo (ej. *“No puedo acceder al sistema de nómina”*), sin trazabilidad ni control. El sistema centraliza y ordena ese flujo:



El sistema conserva todo el historial del ticket.

2. Análisis de Requerimientos

2.1 Actores del Sistema

Actor	Descripción
Usuario (USER)	Empleado que reporta incidencias
Agente (AGENT)	Resuelve tickets asignados
Supervisor (SUPERVISOR)	Gestiona asignaciones y métricas del equipo
Administrador (ADMIN)	Administra catálogos, usuarios y configuración global

2.2 Requerimientos Funcionales

RF-01 — Autenticación y sesión - El sistema debe permitir registrar usuarios, iniciar sesión, refrescar el token de sesión y cerrar sesión.
- El acceso a los recursos debe estar protegido mediante JWT.

RF-02 — Gestión de usuarios (rol USER) - Un usuario debe poder crear tickets, consultar únicamente sus propios tickets, agregar comentarios, cerrar sus tickets y reabrirlos.

RF-03 — Gestión de tickets por agentes (rol AGENT) - Un agente debe poder consultar los tickets que tiene asignados, tomar tickets, cambiar su estado, agregar comentarios y marcarlos como resueltos. - Un agente solo puede modificar tickets que le fueron asignados.

RF-04 — Supervisión (rol SUPERVISOR) - Un supervisor debe poder asignar y reasignar tickets, consultar los tickets del equipo, consultar métricas y gestionar prioridades.

RF-05 — Administración (rol ADMIN) - Un administrador debe poder gestionar usuarios, agentes, categorías, departamentos, políticas de SLA y consultar los registros de auditoría.

RF-06 — Ciclo de vida del ticket - El sistema debe implementar una máquina de estados para el ticket (ver sección 3), impidiendo transiciones no permitidas (ej. no se puede cerrar un ticket que no esté en estado `RESOLVED`).

RF-07 — Comentarios - El sistema debe permitir comentarios públicos (visibles para el usuario) e internos (visibles solo para agentes/supervisores/administradores).

RF-08 — Adjuntos - El sistema debe permitir adjuntar archivos a un ticket (capturas, logs, documentos).

RF-09 — Asignaciones - El sistema debe registrar quién asignó cada ticket, a qué agente, cuándo, y el motivo de una reasignación.

RF-10 — Historial y auditoría - Toda transición de estado y cambio relevante (ej. cambio de prioridad) debe quedar registrado en el historial del ticket. - Las acciones administrativas y de seguridad deben quedar registradas en el log de auditoría.

RF-11 — SLA - El sistema debe asociar cada prioridad a una política de SLA (tiempo de respuesta y de resolución) y calcular la fecha límite (`due_at`) del ticket al crearlo. - Todo ticket de prioridad `CRITICAL` debe tener una política de SLA asociada.

RF-12 — Notificaciones - El sistema debe notificar a los usuarios ante eventos relevantes del ticket (ej. asignación).

RF-13 — Búsqueda y filtrado - El sistema debe permitir filtrar tickets por estado, prioridad, categoría, agente, departamento y fecha, así como realizar búsquedas por número de ticket o título, con soporte de paginación.

RF-14 — Panel de indicadores (dashboard) - El sistema debe exponer métricas agregadas: total de tickets, tickets abiertos/en progreso/resueltos, tickets críticos, incumplimientos de SLA y desempeño por agente.

2.3 Reglas de Negocio

#	Regla
1	Un usuario solamente puede consultar sus propios tickets
2	Un agente solamente puede modificar tickets asignados a él
3	Un supervisor puede reasignar tickets
4	Un ticket <code>CLOSED</code> no puede modificarse, salvo para reabrirlo (<code>REOPEN</code>)
5	Un ticket <code>RESOLVED</code> puede ser reabierto
6	No se puede cerrar un ticket que no esté <code>RESOLVED</code>
7	Un ticket <code>CRITICAL</code> debe tener SLA asociado
8	Un agente inactivo no puede recibir asignaciones de tickets

2.4 Requerimientos No Funcionales

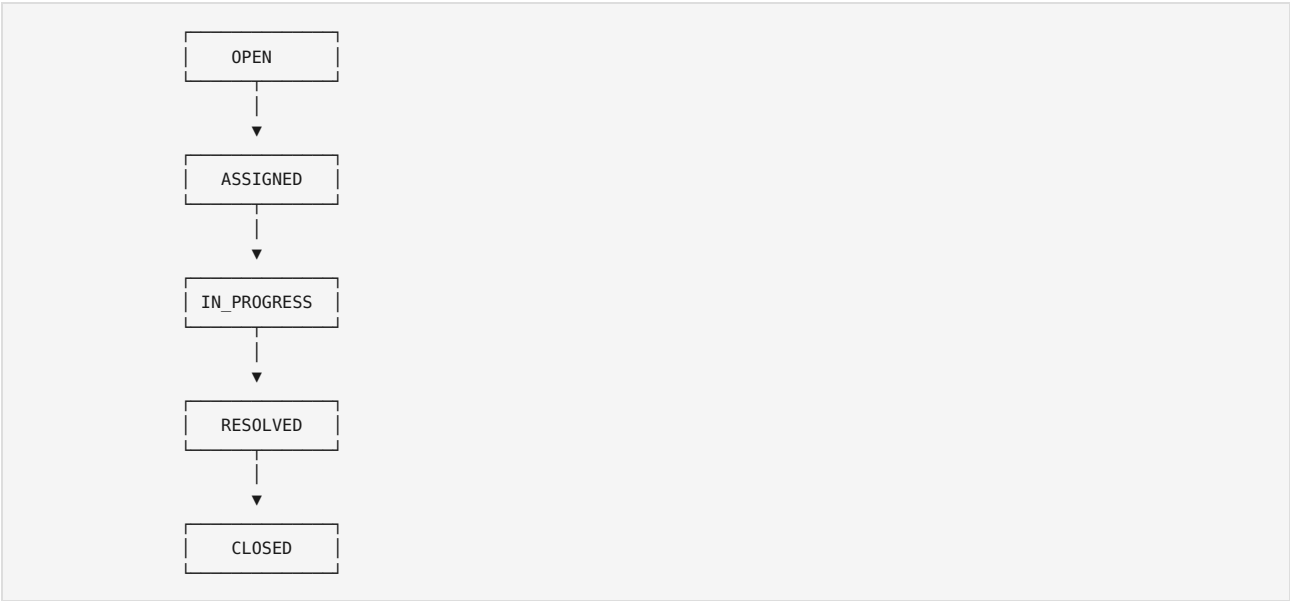
#	Requerimiento	Cómo se cubre en el diseño
RNF-01	Seguridad de acceso	Spring Security + JWT, autorización por rol
RNF-02	Trazabilidad	Tablas <code>ticket_history</code> y <code>audit_logs</code>
RNF-03	Integridad de datos	Migraciones versionadas con Flyway (no <code>ddl-auto</code>)
RNF-04	Rendimiento en listados	Paginación obligatoria (<code>Page<TicketResponse></code>)
RNF-05	Mantenibilidad	Arquitectura modular por dominio, DTOs con MapStruct
RNF-06	Calidad	Pruebas unitarias (JUnit 5 + Mockito) y de integración (Testcontainers)
RNF-07	Documentación	OpenAPI / Swagger para todos los endpoints
RNF-08	Observabilidad	Spring Boot Actuator (<code>/health</code> , <code>/info</code> , <code>/metrics</code>)
RNF-09	Portabilidad / despliegue	Contenedores Docker y Docker Compose
RNF-10	Escalabilidad futura	Diseño preparado para evolucionar a eventos (Kafka) y microservicios

3. Roles y Permisos

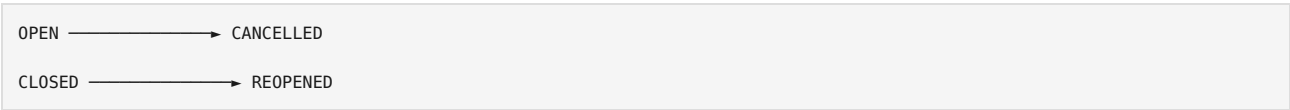
Rol	Permisos
USER	Crear tickets · Consultar sus tickets · Agregar comentarios · Cerrar sus tickets · Reabrir tickets
AGENT	Consultar tickets asignados · Tomar tickets · Cambiar estado · Agregar comentarios · Resolver tickets
SUPERVISOR	Asignar tickets · Reasignar tickets · Consultar tickets del equipo · Consultar métricas · Gestionar prioridades
ADMIN	Gestionar usuarios · Gestionar agentes · Gestionar categorías · Gestionar departamentos · Gestionar SLA · Consultar auditoría

4. Flujo de Estados del Ticket

Flujo principal:



Transiciones adicionales:



Esto permite implementar una verdadera **máquina de estados**, en lugar de cambiar un `status` libremente sin reglas.

5. Prioridades

Prioridad	Ejemplo
LOW	Solicitud de información
MEDIUM	Error que afecta a un usuario
HIGH	Error que afecta a varios usuarios
CRITICAL	Sistema completamente caído

6. Categorías

Categorías principales: `HARDWARE` , `SOFTWARE` , `NETWORK` , `ACCESS` , `SECURITY` , `DATABASE` , `OTHER` .

Ejemplos de subtemas dentro de cada categoría:

ACCESS

- Problemas de contraseña

NETWORK

- VPN
- Internet
- DNS

HARDWARE

- Laptop
- Monitor
- Impresora

7. Departamentos

Departamentos iniciales: IT, FINANCE, HR, SALES, OPERATIONS. Permiten asignar tickets por área de la empresa.

8. Modelo de Base de Datos

8.1 Tablas del Sistema

users · roles · user_roles · departments · categories · tickets · ticket_comments · ticket_attachments · ticket_assignments · ticket_history · sla_policies · notifications · audit_logs

8.2 users

Campo	Tipo
id	UUID PK
first_name	VARCHAR
last_name	VARCHAR
email	VARCHAR UNIQUE
password	VARCHAR
phone	VARCHAR
department_id	UUID FK
active	BOOLEAN
created_at	TIMESTAMP
updated_at	TIMESTAMP

8.3 roles

Campo
id
name

Datos iniciales:

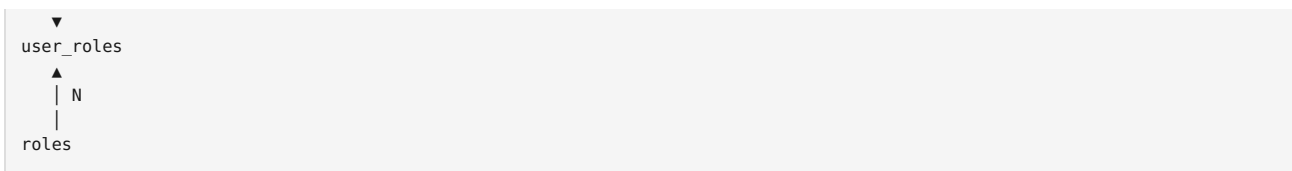
id	name
1	ROLE_USER
2	ROLE_AGENT
3	ROLE_SUPERVISOR
4	ROLE_ADMIN

8.4 user_roles

Relación N a N entre users y roles:

users

- N



Campo
user_id
role_id

Así, un usuario puede tener uno o varios roles.

8.5 departments

Campo
id
name
description
active
created_at
updated_at

Ejemplo: IT, Finance, HR, Sales.

8.6 categories

Campo
id
name
description
department_id
active
created_at
updated_at

8.7 tickets (tabla principal)

Campo	Tipo
id	UUID PK
ticket_number	VARCHAR UNIQUE
title	VARCHAR
description	TEXT
status	VARCHAR
priority	VARCHAR
category_id	UUID FK
department_id	UUID FK
created_by	UUID FK
assigned_to	UUID FK
sla_policy_id	UUID FK
opened_at	TIMESTAMP
assigned_at	TIMESTAMP
resolved_at	TIMESTAMP
closed_at	TIMESTAMP
due_at	TIMESTAMP
created_at	TIMESTAMP
updated_at	TIMESTAMP

ticket_number podría verse así: HD-000001, HD-000002, HD-000003 — aunque internamente se use UUID.

8.8 ticket_comments

Campo
id
ticket_id
user_id

comment
internal
created_at
updated_at

El campo `internal` distingue comentarios públicos (ej. “*Estamos revisando su problema*”) de comentarios internos (ej. “*Revisar logs del servidor XYZ*”), que el usuario no debe poder ver.

8.9 ticket_attachments

Para adjuntar archivos como `captura.png`, `error.log`, `documento.pdf`.

Campo
id
ticket_id
uploaded_by
file_name
file_path
content_type
file_size
created_at

Inicialmente los archivos se guardarían localmente; más adelante se podría migrar a AWS S3.

8.10 ticket_assignments

Campo
id
ticket_id
assigned_to
assigned_by
assigned_at
unassigned_at
reason

Permite responder: ¿quién asignó este ticket?, ¿a qué agente estuvo asignado?, ¿cuándo fue reasignado?

8.11 ticket_history

Campo
id
ticket_id
user_id
action
old_value
new_value
description
created_at

Ejemplo de trazabilidad para el ticket `HD-000123`:

```

USER created ticket
OPEN → ASSIGNED
ASSIGNED → IN_PROGRESS
Priority: MEDIUM → HIGH
IN_PROGRESS → RESOLVED
RESOLVED → CLOSED

```

8.12 sla_policies

Campo
id
name
priority
response_time_minutes
resolution_time_minutes

active
created_at
updated_at

Prioridad	Respuesta	Resolución
LOW	8 horas	48 horas
MEDIUM	4 horas	24 horas
HIGH	1 hora	8 horas
CRITICAL	15 min	2 horas

Cálculo al crear el ticket:

Ticket → Priority = HIGH → SLA HIGH → Resolution = 8 horas → due_at

8.13 notifications

Campo
id
user_id
ticket_id
type
title
message
read
created_at
read_at

Ejemplo: “Ticket HD-000123 fue asignado a ti.” Más adelante se puede integrar con Email, WebSocket o Kafka.

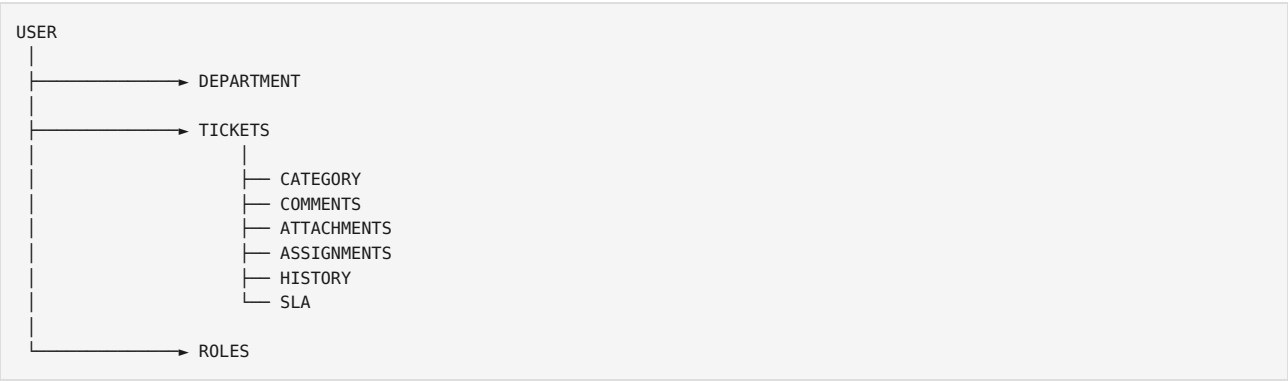
8.14 audit_logs

Campo
id
user_id
action
entity
entity_id
ip_address
details
created_at

Ejemplo:

USER: alex
ACTION: UPDATE_TICKET
ENTITY: TICKET
ENTITY_ID: 123

8.15 Relaciones del Modelo



9. API REST

9.1 Autenticación

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
```

9.2 Usuarios

```
GET /api/v1/users
GET /api/v1/users/{id}
POST /api/v1/users
PUT /api/v1/users/{id}
PATCH /api/v1/users/{id}/status
DELETE /api/v1/users/{id}
```

9.3 Tickets

CRUD base:

```
POST /api/v1/tickets
GET /api/v1/tickets
GET /api/v1/tickets/{id}
PUT /api/v1/tickets/{id}
DELETE /api/v1/tickets/{id}
```

Endpoints de negocio (preferibles a un PUT genérico para modificar todo):

```
PATCH /api/v1/tickets/{id}/assign
PATCH /api/v1/tickets/{id}/start
PATCH /api/v1/tickets/{id}/resolve
PATCH /api/v1/tickets/{id}/close
PATCH /api/v1/tickets/{id}/cancel
PATCH /api/v1/tickets/{id}/reopen
```

9.4 Comentarios

```
POST /api/v1/tickets/{ticketId}/comments
GET /api/v1/tickets/{ticketId}/comments
```

9.5 Historial

```
GET /api/v1/tickets/{ticketId}/history
```

9.6 Asignación

```
PATCH /api/v1/tickets/{ticketId}/assign
```

Request:

```
{
  "agentId": "550e8400-e29b-41d4-a716-446655440000"
}
```

9.7 Filtros

```
GET /api/v1/tickets?status=OPEN
GET /api/v1/tickets?priority=HIGH
```

```
GET /api/v1/tickets?category=NETWORK
GET /api/v1/tickets?status=OPEN&priority=HIGH&category=NETWORK
```

Con paginación y orden:

```
GET /api/v1/tickets?page=0&size=20&sort=createdAt,desc
```

9.8 Búsqueda

Se puede buscar por: número de ticket, título, estado, prioridad, categoría, agente, departamento y fecha de creación.

```
GET /api/v1/tickets?search=VPN&status=OPEN&priority=HIGH
```

Más adelante se puede incorporar `JpaSpecificationExecutor` para filtros dinámicos.

9.9 Dashboard

```
GET /api/v1/dashboard
```

Respuesta general:

```
{
  "totalTickets": 1250,
  "openTickets": 120,
  "inProgressTickets": 80,
  "resolvedTickets": 950,
  "criticalTickets": 12,
  "slaBreached": 7
}
```

Por agente:

```
{
  "agent": "John Doe",
  "assigned": 18,
  "resolved": 42,
  "averageResolutionMinutes": 135
}
```

Esto además deja preparado el terreno para un futuro frontend en React.

10. DTOs

No se exponen entidades JPA directamente:

- `CreateTicketRequest`, `UpdateTicketRequest`, `AssignTicketRequest`, `AddCommentRequest`
- `TicketResponse`, `TicketDetailResponse`, `TicketSummaryResponse`, `CommentResponse`

Mapeo con MapStruct:

```
Entity → MapStruct → DTO
```

11. Arquitectura

Arquitectura **hexagonal (puertos y adaptadores)**, organizada por módulo de negocio. Cada módulo separa `application` (casos de uso y puertos), `domain` (modelo y excepciones de negocio) e `infrastructure.adapter` (adaptadores de entrada REST y de salida a persistencia).

11.1 Paquete base

11.2 Estructura de un módulo (ejemplo: department)



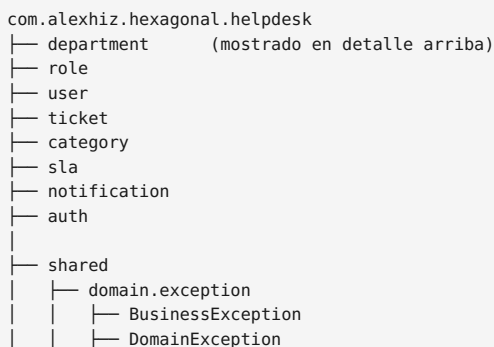
Cómo fluye una petición:

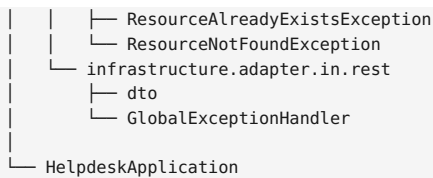


El dominio (`Department`, sus excepciones y las reglas de negocio) no depende de Spring ni de JPA: solo depende de sus propios puertos. Esto permite cambiar de base de datos o de framework de entrada sin tocar la lógica de negocio.

11.3 Módulos del sistema

El mismo patrón (`application` / `domain` / `infrastructure.adapter`) se replica en cada módulo de negocio:





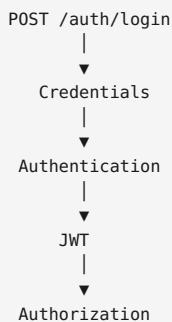
`shared` concentra lo transversal a todos los módulos: excepciones base de dominio (de las que heredan las excepciones específicas, ej. `DepartmentNotFoundException` extends `ResourceNotFoundException`) y el manejo global de errores (`GlobalExceptionHandler`) que traduce esas excepciones a la respuesta estándar de error (ver sección 13).

11.4 Convención de nombres por capa

Capa	Sufijo / patrón	Ejemplo
Caso de uso (puerto de entrada)	<code>...UseCase</code>	<code>CreateDepartmentUseCase</code>
Puerto de salida	<code>...RepositoryPort</code>	<code>DepartmentRepositoryPort</code>
Implementación del caso de uso	<code>...Service</code>	<code>DepartmentService</code>
Modelo de dominio	Nombre de la entidad	<code>Department</code>
Excepción de dominio	<code>...AlreadyExistsException</code> / <code>...NotFoundException</code>	<code>DepartmentNotFoundException</code>
Adaptador de entrada REST	<code>...Controller</code>	<code>DepartmentController</code>
DTOs REST	<code>...Request</code> / <code>...Response</code>	<code>DepartmentRequest</code> , <code>DepartmentResponse</code>
Entidad de persistencia	<code>...Entity</code>	<code>DepartmentEntity</code>
Adaptador de salida (implementa el puerto)	<code>...PersistenceAdapter</code>	<code>DepartmentPersistenceAdapter</code>
Mapper Entity <-> Domain	<code>...PersistenceMapper</code>	<code>DepartmentPersistenceMapper</code>
Repositorio Spring Data	<code>...Repository</code>	<code>DepartmentRepository</code>

12. Seguridad

Spring Security + JWT:



Ejemplos de autorización por rol:

```

USER      → GET /tickets/my
AGENT     → GET /tickets/assigned
SUPERVISOR → GET /tickets/team
ADMIN     → *

```

13. Manejo de Errores

Con `@RestControllerAdvice` y excepciones específicas:

- `TicketNotFoundException`
- `UserNotFoundException`
- `CategoryNotFoundException`
- `InvalidTicketStateException`
- `UnauthorizedTicketAccessException`
- `AgentNotAvailableException`
- `SlaNotFoundException`

Respuesta estándar de error:

```
{
  "timestamp": "2026-08-19T11:30:00",
  "status": 404,
  "code": "TICKET_NOT_FOUND",
  "message": "Ticket HD-000123 was not found",
  "path": "/api/v1/tickets/123"
}
```

14. Validaciones

```
public record CreateTicketRequest(

    @NotBlank
    @Size(max = 150)
    String title,

    @NotBlank
    String description,

    @NotNull
    UUID categoryId,

    @NotNull
    Priority priority

) {}
```

15. Transacciones

Ejemplo de operación que debe analizarse con `@Transactional`:

```
Crear Ticket
├── Guardar Ticket
├── Crear History
└── Crear Notification
```

16. Paginación

Evitar respuestas como `GET /tickets` devolviendo 500,000 registros. Usar `Page<TicketResponse>`:

```
GET /api/v1/tickets?page=0&size=20
```

17. Pruebas (Tests)

17.1 Pruebas Unitarias

Con JUnit 5 + Mockito:

- `TicketServiceTest`
- `TicketAssignmentServiceTest`
- `TicketStateServiceTest`
- `SlasServiceTest`

17.2 Pruebas de Integración

Con Testcontainers + PostgreSQL:

- `TicketControllerIntegrationTest`
- `AuthIntegrationTest`
- `TicketRepositoryIntegrationTest`

18. Docker

Etapas inicial:

```
helpdesk-api
postgres
```

Segunda etapa:

```
helpdesk-api
postgres
redis
```

Tercera etapa:

```
helpdesk-api
postgres
redis
kafka
zookeeper / kafka kraft
```

19. Flyway

No se depende de `spring.jpa.hibernate.ddl-auto: update`. Se usan migraciones versionadas:

```
V1__create_users.sql
V2__create_roles.sql
V3__create_departments.sql
V4__create_categories.sql
V5__create_tickets.sql
V6__create_ticket_comments.sql
V7__create_ticket_history.sql
V8__create_sla_policies.sql
```

20. Swagger

Documentación de:

```
/api/v1/auth/**
/api/v1/users/**
/api/v1/tickets/**
/api/v1/categories/**
/api/v1/departments/**
/api/v1/sla/**
```

Con autenticación Bearer:

```
Authorization: Bearer <JWT>
```

21. Actuator

```
/actuator/health
/actuator/info
/actuator/metrics
```

22. Logging

Con SLF4J (nunca `System.out.println()`):

```
log.info("Creating ticket for user {}", userId);
log.warn("Ticket {} exceeded SLA", ticketId);
log.error("Error processing ticket {}", ticketId, exception);
```

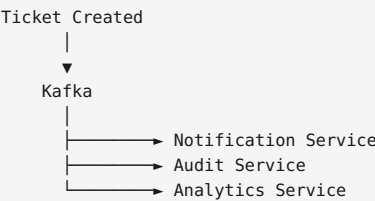
23. Roadmap de Evolución

23.1 Segunda Etapa: Redis

Cache para `Categorías`, `Departamentos`, `SLA`, `Usuarios` y para `GET /tickets/{id}`.

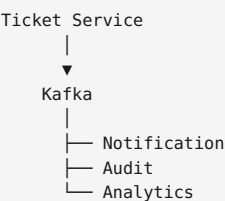
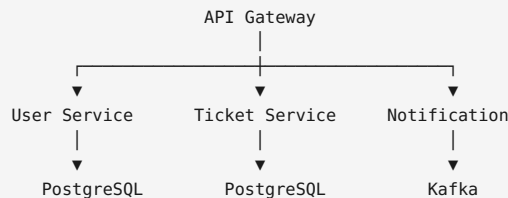
23.2 Tercera Etapa: Eventos

```
TicketCreated
TicketAssigned
TicketResolved
TicketClosed
TicketReopened
SlaExceeded
```



Esto conecta con conceptos de arquitectura orientada a eventos: Kafka, desacoplamiento y, eventualmente, patrón Outbox.

23.3 Cuarta Etapa: Microservicios



24. Estructura Final del Proyecto

```
helpdesk-api/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com.example.helpdesk/
│   │   └── resources/
│   │       ├── application.yml
│   │       └── db/
│   │           └── migration/
│   └── test/
├── docker/
│   └── Dockerfile
├── docker-compose.yml
├── pom.xml
├── README.md
└── .gitignore
```